Lab Handout 4-C

DummyClassifier, Decision Tree, and Random Forest with Grid Search

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Establish a three-rung classical baseline for each of the four classification tasks: a majority-class `DummyClassifier`, a single `DecisionTreeClassifier` with fixed depth, and a grid-searched `RandomForestClassifi` Every subsequent model in Module 4 must beat the random-forest number; if a neural network cannot outperform the forest, the feature representation — not the model family — is the problem.

2 Prerequisites

- Lab Handout 4-B complete (`Code/AIML/artifacts/{sf,mod,beacon,pkt}/` present).
- `scikit-learn` $\geq$ `1.3` and `matplotlib` installed.
- `scipy` (already required by scikit-learn) is used for the dependency-free Wilson CI in Step 4.

3 Background: Why These Three Baselines?

- **DummyClassifier** (majority class) establishes the *lower bound*. Any model below this number is actively harmful.
- **Single decision tree** (`max_depth=5`) gives a *legible* model whose splits you can read. If the chosen feature on the root split makes no physical sense, that is an early warning sign.
- **Random forest** with moderate grid search is the *strong classical benchmark*. Tree ensembles are the best-performing classical family on tabular RF feature data in the published literature; beating them with a neural network on a few thousand samples is the harder direction.

4 Procedure

4.1 Step 1 — Logging setup and the common loader

The script (`Code/AIML/baselines.py`) follows the same logging pattern as LH4-A and LH4-B: stdout plus a timestamped archive log and a stable `baselines_latest.log` in `Code/AIML/Logs/`.

```python
import json
import logging
import sys
from datetime import datetime, timezone
from math import sqrt
```

```python
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

SCRIPT_DIR    = Path(__file__).parent
LOG_DIR       = SCRIPT_DIR / "Logs"
ARTIFACTS_DIR = SCRIPT_DIR / "artifacts"
FIGS_DIR      = SCRIPT_DIR / "figs"
MANIFEST_PATH = SCRIPT_DIR / "baselines_manifest.json"

LOG_DIR.mkdir(parents=True, exist_ok=True)
FIGS_DIR.mkdir(parents=True, exist_ok=True)

run_stamp   = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
log_archive = LOG_DIR / f"baselines_{run_stamp}.log"
log_latest  = LOG_DIR / "baselines_latest.log"

log = logging.getLogger("baselines")
log.setLevel(logging.DEBUG)
log.handlers.clear()
fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s",
                        datefmt="%Y-%m-%d %H:%M:%S")
for h in (logging.StreamHandler(sys.stdout),
          logging.FileHandler(log_archive, mode="w", encoding="utf-8"),
          logging.FileHandler(log_latest,  mode="w", encoding="utf-8")):
    h.setFormatter(fmt)
    log.addHandler(h)

log.info("=== baselines.py starting ===")
log.info(f"Run stamp: {run_stamp}")


def load_task(task):
    d = ARTIFACTS_DIR / task
    X_tr = np.load(d / "X_train.npy"); y_tr = np.load(d / "y_train.npy")
    X_va = np.load(d / "X_val.npy");   y_va = np.load(d / "y_val.npy")
    X_te = np.load(d / "X_test.npy");  y_te = np.load(d / "y_test.npy")
    pre  = joblib.load(d / "preprocessor.pkl")
    return X_tr, y_tr, X_va, y_va, X_te, y_te, pre
```

Listing 1: Top of `baselines.py` — imports, paths, logging, and the common loader.

4.2 Step 2 — Wilson score confidence interval

Every accuracy reported in Step 3 ships with a 95% Wilson interval, not a standard-normal interval (which can produce negative probabilities for small n or extreme p). The implementation uses scipy only — no external CI library required.

```
def wilson_ci(n_correct, n_total, alpha=0.05):
    if n_total == 0:
        return 0.0, 1.0
    p = n_correct / n_total
    z = norm.ppf(1 - alpha / 2)
    denom = 1 + z * z / n_total
    center = (p + z * z / (2 * n_total)) / denom
    half = z * sqrt(p * (1 - p) / n_total
                    + z * z / (4 * n_total * n_total)) / denom
    return max(0.0, center - half), min(1.0, center + half)


def report_acc(label, y_true, y_pred):
    n = len(y_true)
    n_correct = int((y_pred == y_true).sum())
    acc = n_correct / n if n else 0.0
    lo, hi = wilson_ci(n_correct, n)
    log.info(f"  {label}: acc = {acc:.3f}  [95% CI {lo:.3f}, {hi:.3f}]  "
             f"({n_correct}/{n})")
    return {"acc": acc, "ci_lo": lo, "ci_hi": hi,
            "n_correct": n_correct, "n_total": n}
```

4.3 Step 3 — Run the three baselines per task

For each task, fit the dummy, depth-5 decision tree, and grid-searched random forest. Single-class tasks (the `mod` task in the current C1+C2+C5 dataset is LoRa-only) are skipped because none of these classifiers are defined for a single class.

```
RF_GRID = {
    "n_estimators":     [100, 300, 600],
    "max_depth":        [None, 10, 20],
    "min_samples_leaf": [1, 3],
    "max_features":     ["sqrt", "log2"],
}

results = {}
for task in ["sf", "mod", "beacon", "pkt"]:
    log.info(f"--- Task: {task} ---")
    X_tr, y_tr, X_va, y_va, X_te, y_te, pre = load_task(task)
    n_classes = len(np.unique(y_tr))
    log.info(f"Train shape={X_tr.shape}, classes={n_classes}, "
             f"features={pre['feature_cols']}")
    if n_classes < 2:
        log.warning(f"{task}: only {n_classes} class --- skipping
            baselines")
        results[task] = {"skipped": True, "reason": "single class in train
            "}
        continue
```

```python

    # Rung 1: Dummy
    dum = DummyClassifier(strategy="most_frequent", random_state=42)
    dum.fit(X_tr, y_tr)
    dum_res = report_acc("Dummy", y_te, dum.predict(X_te))

    # Rung 2: Single decision tree, depth 5
    dt = DecisionTreeClassifier(max_depth=5, random_state=42)
    dt.fit(X_tr, y_tr)
    dt_res = report_acc("Tree-5", y_te, dt.predict(X_te))

    # Rung 3: Random forest grid search on train+val (test is held out).
    X_cv = np.concatenate([X_tr, X_va]); y_cv = np.concatenate([y_tr, y_va])
    rf = RandomForestClassifier(random_state=42, n_jobs=-1)
    gs = GridSearchCV(rf, RF_GRID, cv=5, scoring="accuracy",
                      n_jobs=-1, verbose=0)
    log.info(f"  RF grid search: {len(RF_GRID['n_estimators']) * "
             f"len(RF_GRID['max_depth']) * len(RF_GRID['min_samples_leaf']) "
             f"* len(RF_GRID['max_features'])} combos x 5 folds...")
    gs.fit(X_cv, y_cv)
    best = gs.best_estimator_
    rf_res = report_acc("RF best", y_te, best.predict(X_te))
    rf_res["cv_acc"] = float(gs.best_score_)
    rf_res["params"] = gs.best_params_
    log.info(f"  RF best params: {gs.best_params_}")
    log.info(f"  RF CV acc = {gs.best_score_:.3f}")

    # Save the winning forest and its metadata.
    joblib.dump({"model": best, **rf_res},
                ARTIFACTS_DIR / task / "rf.pkl")
    log.info(f"  Saved {ARTIFACTS_DIR / task / 'rf.pkl'}")

    results[task] = {"dummy": dum_res, "tree5": dt_res, "rf": rf_res,
                     "feature_cols": pre["feature_cols"]}
```

Why 5-Fold Stratified CV?

With ∼2,200 samples split 70/15/15, the validation set is only ∼200 rows. Hyperparameter choices tuned against a set that small are noisy. Concatenating train and val for a 5-fold stratified cross-validation averages over five independent validation folds, giving each hyperparameter combination an effective sample size of the entire non-test data. The test split remains genuinely untouched for final reporting.

4.4 Step 4 — Feature importance from the best forest

```python
def plot_importance(model, feature_names, task):
    imp = model.feature_importances_
    order = np.argsort(imp)[::-1]
    plt.figure(figsize=(7, 3.5))
    plt.bar(range(len(imp)), imp[order], color="steelblue")
```

```
    plt.xticks(range(len(imp)),
               [feature_names[i] for i in order], rotation=30, ha="right")
    plt.ylabel("Gini importance")
    plt.title(f"Random Forest feature importance --- task: {task}")
    plt.tight_layout()
    out = FIGS_DIR / f"rf_importance_{task}.png"
    plt.savefig(out, dpi=150)
    plt.close()
    log.info(f"  Saved {out}")
    return [(feature_names[i], float(imp[i])) for i in order]


for task, res in results.items():
    if res.get("skipped"):
        continue
    rf_pkl = joblib.load(ARTIFACTS_DIR / task / "rf.pkl")
    ranking = plot_importance(rf_pkl["model"], res["feature_cols"], task)
    res["feature_importance"] = ranking
    log.info(f"  Top features ({task}): "
             f"{', '.join(f'{n}={v:.2f}' for n, v in ranking[:3])}")
```

For the SF task in the current dataset, `snr_db` and `rssi_dbm` are expected to dominate (SF processing gain shows up most strongly in SNR). For emitter ID, `inter_arrival_ms` typically dominates because Target B's ACK cadence is bimodal versus Target A's clean 1 s/2 s tick. For packet type, `inter_arrival_ms` also dominates (ACK packets follow DATA packets within a tight latency window). Any wildly different ordering is worth investigating.

4.5 Step 5 — Summary table and manifest

Print a populated summary table to the log so the deliverable is self-documenting, and write `baselines_manifest.json` so LH4-D through LH4-F can read these numbers without re-fitting.

```
log.info("=== Baseline Summary ===")
log.info(f"{'Task':<8} {'Dummy':>10} {'Tree-5':>10} {'RF best':>10} "
         f"{'RF 95% CI':>20}")
for task in ["sf", "mod", "beacon", "pkt"]:
    res = results[task]
    if res.get("skipped"):
        log.info(f"{task:<8} {'(skipped)':>10}  reason: {res['reason']}")
        continue
    d = res["dummy"]["acc"]; t = res["tree5"]["acc"]; r = res["rf"]
    log.info(f"{task:<8} {d:>10.3f} {t:>10.3f} {r['acc']:>10.3f} "
             f"   [{r['ci_lo']:.3f}, {r['ci_hi']:.3f}]")

manifest = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "run_stamp":    run_stamp,
    "log_archive":  str(log_archive),
    "rf_grid":      RF_GRID,
    "results":      results,
}
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2, default=str)
log.info(f"Wrote {MANIFEST_PATH}")
```

```
log.info("=== baselines.py done ===")
```

The summary table that the script prints is the same shape as the deliverable below; carry the populated values forward to LH4-D through LH4-F. Every model in those handouts is benchmarked against the RF column.

Task	Dummy	Tree-5	RF best	Wilson 95% CI (RF)
SF Detection				
Modulation	(skipped — only LoRa collected; revisit once C3 or other modulations exist)			
Emitter ID				
Packet Type				

Table 1: Baselines from Steps 3–4. Fill from the script's printed summary or from `baselines_manifest.json`.

5 Verification Checklist

- ☐ Dummy, tree, and random-forest accuracies computed for every task with ≥2 classes (`sf`, `beacon`, `pkt` in the current dataset; `mod` skipped with logged reason).
- ☐ Grid search complete; best parameters recorded in `rf.pkl` and the manifest.
- ☐ Wilson 95% CI reported for every accuracy (Dummy, Tree-5, RF).
- ☐ `Code/AIML/figs/rf_importance_<task>.png` produced for each non-skipped task.
- ☐ `Code/AIML/artifacts/<task>/rf.pkl` saved for each non-skipped task.
- ☐ `Code/AIML/baselines_manifest.json` written.
- ☐ `Code/AIML/Logs/baselines_<timestamp>.log` archived.

Lab Handout 4-C Deliverable

1. `Code/AIML/baselines.py`: the runnable script.
2. `Code/AIML/baselines_manifest.json`: per-task Dummy/Tree-5/RF accuracies, Wilson CIs, RF best-params, and feature-importance ranking.
3. Filled baseline summary table (rows for non-skipped tasks; `mod` row marked skipped).
4. Three feature-importance bar plots (`sf`, `beacon`, `pkt`), each with a 2–3 sentence interpretation.
5. `Code/AIML/artifacts/{sf,beacon,pkt}/rf.pkl`: the grid-searched random forests.
6. One short paragraph per non-skipped task discussing whether the top-ranked features match the physical expectation (e.g., SNR dominating SF detection; `inter_arrival_ms` dominating packet type and emitter ID).

7. `Code/AIML/Logs/baselines_<timestamp>.log`: archived audit trail showing the populated summary table.